

ASSIGNMENT 03

Title : Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not in CloudSim

Name :- Shinde Ajayraje Dilipkumar

Rollno :- 23107118

Branch :- AI&DS

Code (PriorityScheduling.java) :-

```
import java.util.*;
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.*;

public class PriorityScheduling {

    public static void main(String[] args) {

        try {
            CloudSim.init(1, Calendar.getInstance(), false);

            Datacenter dc = createDatacenter("DC");
            DatacenterBroker broker = new DatacenterBroker("Broker");
            int id = broker.getId();

            // Create 2 VMs
            List<Vm> vmList = new ArrayList<>();
            for (int i = 0; i < 2; i++) {
                vmList.add(new Vm(i, id, 1000, 1, 512, 1000, 10000, "Xen",
                    new CloudletSchedulerTimeShared()));
            }
            broker.submitVmList(vmList);

            // Create Cloudlets with priority
            List<PriorityCloudlet> list = new ArrayList<>();
            list.add(new PriorityCloudlet(0, 4000, 3));
            list.add(new PriorityCloudlet(1, 2000, 1));
            list.add(new PriorityCloudlet(2, 3000, 5));
            list.add(new PriorityCloudlet(3, 1000, 2));

            // Sort by priority (HIGH → LOW)
            Collections.sort(list, (c1, c2) -> c2.priority - c1.priority);

            // Convert to normal cloudlet list
            List<Cloudlet> finalList = new ArrayList<>();
            for (PriorityCloudlet c : list) {
                c.setUserId(id);
                finalList.add(c);
            }

            broker.submitCloudletList(finalList);
```

```

        CloudSim.startSimulation();
        List<Cloudlet> result = broker.getCloudletReceivedList();
        CloudSim.stopSimulation();

        print(result);

    } catch (Exception e) {
        e.printStackTrace();
    }
}

static class PriorityCloudlet extends Cloudlet {
    int priority;

    PriorityCloudlet(int id, long length, int priority) {
        super(id, length, 1, 300, 300,
            new UtilizationModelFull(),
            new UtilizationModelFull(),
            new UtilizationModelFull());
        this.priority = priority;
    }
}

static Datacenter createDatacenter(String name) throws Exception {
    List<Pe> peList = Arrays.asList(new Pe(0, new PeProvisionerSimple(1000)));

    Host host = new Host(0,
        new RamProvisionerSimple(2048),
        new BwProvisionerSimple(10000),
        1000000,
        peList,
        new VmSchedulerTimeShared(peList));

    DatacenterCharacteristics ch = new DatacenterCharacteristics(
        "x86", "Linux", "Xen",
        Arrays.asList(host), 10.0, 3.0, 0.05, 0.001, 0.0);

    return new Datacenter(name, ch,
        new VmAllocationPolicySimple(Arrays.asList(host)),
        new LinkedList<Storage>(), 0);
}

static void print(List<Cloudlet> list) {
    System.out.println("\n===== OUTPUT =====");
    for (Cloudlet c : list) {
        System.out.println("Cloudlet " + c.getCloudletId()
            + " executed on VM " + c.getVmId());
    }
}
}

```

OUTPUT :-

===== OUTPUT =====

Cloudlet 2 executed on VM 0

Cloudlet 0 executed on VM 1

Cloudlet 3 executed on VM 0

Cloudlet 1 executed on VM 1